

代码字体与语法高亮示例

下面两段使用相同代码和配色, 可以直接比较字体。新版采用接近原稿的 Latin Modern Mono; 中文注释使用 Fandol 仿宋风格。纯文本和目录树继续使用 DejaVu, 以完整显示线条和特殊空格。

新版程序字体: Latin Modern Mono

```
// 时序逻辑: 低电平复位, 保留 0/0、1/1、下划线和引号的区别
module counter(input wire clk, reset_n, output reg [7:0] count);
    always @(posedge clk) begin
        if (!reset_n) count <= 8'h00;
        else count <= count + 1'b1;
    end
    initial $display("counter is ready");
endmodule
```

备选等宽字体: DejaVu Sans Mono (同配色对照)

```
// 时序逻辑: 低电平复位, 保留 0/0、1/1、下划线和引号的区别
module counter(input wire clk, reset_n, output reg [7:0] count);
    always @(posedge clk) begin
        if (!reset_n) count <= 8'h00;
        else count <= count + 1'b1;
    end
    initial $display("counter is ready");
endmodule
```

目录树与特殊字符

```
chip
├─ rtl  处理器设计
│   ├── core.sv
│   └─ soc.sv
└─ sim  仿真环境
ASCII: 0 0 1 l I _ - -- ' " $ # % & < >
```

通用语言与硬件设计

C：关键字、注释、字符串

```
#include <stdint.h>
// 中文注释应完整显示，并与英文注释保持同色。
uint32_t read_value(volatile uint32_t *address) {
    if (address == NULL) return 0;
    printf("value = %u\n", *address);
    return *address;
}
```

Scala / Chisel / SpinalHDL

```
val count = RegInit(0.U(8.W))
when (io.enable) {
    count := count + 1.U
}.otherwise {
    count := 0.U
}
```

SystemVerilog

```
logic [31:0] data;
always_ff @(posedge clk) begin
    if (!reset_n) data <= '0;
    else data <= next_data;
end
```

Shell

```
# 编译命令保持原始内容，不由 LaTeX 执行。
export ARCH=loongarch
make -j$(nproc)
echo "Build completed"
```

本书专用语言规则

LoongArch 汇编

```
.text
.globl reset_entry
reset_entry:
    li.w    $t0, 0x1c000000 # 初始化地址
    ld.w    $a0, $t0, 0
    addi.w  $a0, $a0, 1
    st.w    $a0, $t0, 0
    ertn
```

设备树 DTS

```
uart0: serial@1fe001e0 {
    compatible = "ns16550a";
    reg = <0x1fe001e0 0x8>;
    interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
    status = "okay";
};
```

EDA Tcl

```
# 创建时钟并设置输入延迟。
set target_library "typical.db"
read_verilog design.v
create_clock -name clk -period 10 [get_ports clk]
set_input_delay -clock clk 3 [all_inputs]
compile -map_effort high
```

配置、构建与补丁

Kconfig / defconfig

```
config MEGA_SOC_FPGA
    bool "MEGA SoC for FPGA"
    default y
# CONFIG_DEBUG_INFO is not set
CONFIG_LOG_BUF_SHIFT=18
```

Makefile

```
CC = loongarch32r-linux-gnustf-gcc
all: kernel.o
kernel.o: kernel.c
    $(CC) -c kernel.c -o kernel.o
```

链接脚本

```
OUTPUT_FORMAT(elf32-loongarch)
MEMORY {
    ram (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}
INCLUDE "section.ld"
```

本书新增的高亮语言

C++ 仿真模型

```
// 模型调用示例，保留类型、作用域符号与字符串。
class Model final {
public:
    uint32_t value = 0;
    void step() { value += 1; }
};

int main() {
    Model model;
    model.step();
    std::cout << "value = " << model.value << std::endl;
}
```

JSON 配置（支持原稿的注释写法）

```
{
    // 解码器配置片段；原稿采用带注释的 JSON 示例。
    "module_name": "core_decoder",
    "enabled": true,
    "width": 32
}
```

C Shell / 签核脚本

```
#!/bin/csh -f
# 保留与 Bash 不同的变量赋值写法。
set top = design
set gds = /design_path/design.gds.gz
if ( -e $gds ) then
    echo "Input exists: $gds"
endif
```